Day 9 - OOP Advanced: Inheritance, Prototypes, Static Members

Day 9 — OOP Advanced: Inheritance, Prototypes, Static Members

Objectives

- Use `extends` and `super` to build one class on top of another
- Understand polymorphism in JavaScript
- Understand what's actually happening underneath `class` — JavaScript's prototype system (this is genuinely JS-specific and worth knowing)
- Understand static methods/properties — things that belong to the class itself, not to any one instance

Inheritance with `extends` and `super`

```
class Animal {
    constructor(name) {
        this.name = name;
    }

    speak() {
        throw new Error("Subclasses must implement speak()");
    }

    introduce() {
        return `I am ${this.name} and I say ${this.speak()}`;
    }
}

class Dog extends Animal {
    speak() {
        return "Woof";
    }
}

class Cat extends Animal {
    speak() {
        return "Meow";
    }
}

const rex = new Dog("Rex");
console.log(rex.introduce());   // I am Rex and I say Woof
```

`class Dog extends Animal` means `Dog` automatically gets everything `Animal` has (its constructor, `introduce()`), and only needs to define what's genuinely different about a `Dog` — here, just `speak()`. `Animal.speak()` deliberately `throw`s, meaning "every subclass MUST provide its own version" — if a new

subclass forgot to define `speak()`, calling it would immediately, clearly fail, rather than silently doing something wrong.

Notice `introduce()` calls `this.speak()` without knowing in advance whether `this` will be a `Dog`, a `Cat`, or some future subclass — it automatically gets the correct version for whatever object it's actually attached to. **This is polymorphism** ("many forms"): one piece of code behaving differently depending on the actual object it's working with, decided while the program runs, not fixed in advance.

`super` — reaching the parent class's version of something

If a subclass needs its own constructor but still wants the parent's setup logic to run too, call it explicitly with `super(...)`:

```
class Employee extends Animal {   // deliberately silly example, purely to show the mechanism
    constructor(name, salary) {
        super(name);              // runs Animal's constructor with `name`
        this.salary = salary;
    }
}

const e = new Employee("Ana", 50000);
console.log(e.name, e.salary);   // Ana 50000
```

A JavaScript-specific rule worth knowing: if a subclass defines its own constructor, it MUST call `super(...)` before it can use `this` at all — JavaScript enforces this and will throw an error if you try to use `this` before calling `super()`. This makes sense once you know why: the parent class's constructor is what actually finishes setting up the object in the first place; `this` isn't fully ready to use until that's happened.

`super.someMethod()` (not just `super(...)`) also lets you call the *parent's* version of a method you've overridden, from inside the overriding method — useful when you want to add behavior on top of the parent's, rather than fully replacing it.

What's actually happening underneath `class` — prototypes

This section is genuinely specific to JavaScript, and worth knowing because you'll encounter the word "prototype" constantly in JavaScript documentation and interview questions. `class` syntax (which you've been using all of today and yesterday) is, under the hood, a cleaner, more familiar-looking way of writing something JavaScript has always done differently from most other object-oriented languages: instead of classes being their own separate concept, every JavaScript object has a hidden internal link to another object, called its **prototype**, and when you access a property or method that doesn't exist directly on an object, JavaScript automatically looks it up on that object's prototype instead, and so on up a chain, until it finds it or runs out of prototypes to check (this is called the "prototype chain").

```
class Dog {
    bark() {
        return "Woof";
    }
}

const rex = new Dog();
console.log(rex.hasOwnProperty("bark"));                // false -- bark is NOT directly on rex its
console.log(Object.getPrototypeOf(rex) === Dog.prototype);   // true -- rex's prototype is Dog.proto
```

Every instance of `Dog` shares the exact same single `bark` method, living once on `Dog.prototype`, rather than each instance carrying its own separate copy — this is more memory-efficient, and it's the actual mechanism `class` syntax is quietly using for you the whole time. You don't need to manually work with prototypes yourself in modern JavaScript — `class`/`extends` is the clean, modern syntax for everything you'd otherwise have to do manually — but recognizing the word "prototype" and knowing roughly what it refers to (where shared methods actually live, and how JavaScript looks up properties that aren't found directly on an object) will make a lot of JavaScript documentation and error messages (like `TypeError: X.prototype.y is not a function`) make much more sense.

Static methods and properties — belonging to the class itself, not to any instance

Sometimes a piece of data or a function logically belongs to the *class as a whole*, rather than to any one specific instance — for this, use `static`:

```
class Circle {
    static PI = 3.14159;    // a STATIC property -- one single value, shared by the class itself, no

    constructor(radius) {
        this.radius = radius;
    }

    area() {
        return Circle.PI * this.radius ** 2;    // accessed via the CLASS name, not `this`
    }

    static fromDiameter(diameter) {    // a STATIC method -- a kind of alternate way to construct a
        return new Circle(diameter / 2);
    }
}

console.log(Circle.PI);                          // 3.14159 -- accessed directly on the class, no insta
const c = Circle.fromDiameter(10);                  // creates a Circle with radius 5, via the static
console.log(c.area());                                 // uses Circle.PI internally
```

You call a static member on the class itself (`Circle.PI`, `Circle.fromDiameter(...)`) — never on an instance (`c.PI` would be `undefined`, since `PI` doesn't belong to any particular instance). Static methods are commonly used exactly like `fromDiameter` above: as an alternative, named way to construct an instance, when a single constructor isn't expressive enough to cover every way you might want to create one.

Getters/setters revisited, briefly, in the context of inheritance

The `get`/`set` syntax from Day 8 works exactly the same way in a subclass, and can be overridden just like any other method — a subclass can provide its own version of a getter/setter its parent defined, following the exact same polymorphism rules as `speak()` above.

Exercises

Open `exercises.js`, fill in each `// TODO`, and run `node exercises.js`.